From Scratch:
The Design and Implementation of a SAT Solver From Scratch

Oliver Lie

June 18, 2026

1 Abstract

This paper discusses the design and implementation methods of a SAT solver. It explores... INSERT METHODS HERE

2 Introduction

SAT solvers are amazing programs with one simple purpose: solve boolean algebra equations quickly.

2.1 CNF

CNF stands for "Conjunctive Normal Form," sometimes called "Clausal Normal Form." An equation in this form is a "conjunction of disjunctions," ie, it is an equation of "ands" ($\wedge$) in between clauses of "ors" ($\vee$). Each clause of "ors" contains one or more literals, or variables. Each literal has the option to be negated ($\neg$), however, no negation can appear in front of a clause itself. Some examples of CNF equations includes:

- $(A \vee B)$
- $(A \vee B) \wedge (\neg C)$
- $(A \vee B) \wedge (\neg C \vee D)$

Some examples that are not CNF:

- $\neg(A \vee B)$, where there is a NOT in front of a clause
- $\neg(A \vee B) \wedge (C)$, where is a NOT in front of a clause
- $(A \vee B \wedge C)$, where there is an AND within a clause
- $(A \wedge B) \vee (C)$, where it is in DNF form (read ahead)

2.2 DNF

DNF stands for "Disjunctive Normal Form." Just like in CNF form, a DNF equation has only three operators: conjunction ($\wedge$), disjunction ($\vee$), and negation ($\neg$). A DNF equation is in the form of a "disjunction of conjunctions," ie, it is an equation of "ors" ($\vee$) in between clauses of "ands" ($\wedge$). Each clause of "ands" contains one or more literals, each with the option of being negated ($\neg$), with no negation being in front of the clauses themselves. Some examples of DNF equations include:

- $(A \wedge B)$
- $(A \wedge B) \vee (C)$
- $(A \wedge B \wedge C) \vee (D \wedge E)$

Some examples that are not DNF:

- $(A \vee B)$, this is CNF form
- $\neg(A \wedge B) \vee (C)$, where there is a NOT in front of a clause
- $(A \wedge B \vee C) \vee (D \wedge E)$, where there is an OR within a clause

2.3 Defining our Input

For this SAT solver, we will use the DIMACS CNF format of input. DIMACS CNF format consists of seven rules:[1]

1. The file may begin with comment lines. The first character of each comment line must be a lower case letter "c". Comment lines typically occur in one section at the beginning of the file, but are allowed to appear throughout the file.
2. The comment lines are followed by the "problem" line. This begins with a lower case "p" followed by a space, followed by the problem type, which for CNF files is "cnf", followed by the number of variables followed by the number of clauses.
3. The remainder of the file contains lines defining the clauses, one by one.
4. A clause is defined by listing the index of each positive literal, and the negative index of each negative literal. Indices are 1-based, and for obvious reasons the index 0 is not allowed.
5. The definition of a clause may extend beyond a single line of text.
6. The definition of a clause is terminated by a final value of "0".
7. The file terminates after the last clause is defined.

There are some things about this syntax to be wary of:

1. The definition of the next clause normally begins on a new line, but may follow, on the same line, the "0" that marks the end of the previous clause.

[1] https://people.sc.fsu.edu/ jburkardt/data/cnf/cnf.html

2. In some examples of CNF files, the definition of the last clause is not terminated by a final '0'

3. In some examples of CNF files, the rule that the variables are numbered from 1 to N is not followed. The file might declare that there are 10 variables, for instance, but allow them to be numbered 2 through 11.

4. Comments can appear anywhere in the file, but the comment must be on its own line and it must start with 'c' [2]

For a quick example, here is a simple way to format a DIMACS CNF equation:

$$\begin{array}{l} \text{p cnf 4 2} \\ 1\ 2\ -3\ 0 \\ -1\ 4\ 0 \end{array}$$

And one with comments:

$$\begin{array}{l} \text{c this is a comment} \\ \text{p cnf 4 2} \\ \text{c this is another comment } 1\ 2\ -3\ 0 \\ -1\ 4\ 0 \end{array}$$

2.4 Input Processing

In order to process our input, we will provide a few APIs. We will firstly, disallow the header line. Ie, if no header is passed in, rather than throwing an error, we will simply infer the number of variables and clauses. We will also be very loose in terms of what input is considered valid. Our rules will be as follows:

1. Comments can appear anywhere in the file, but they MUST be on their own line and start with 'c'.

2. A header is not necessary, and if one is not provided, then the number of variables and clauses shall be inferred. However, if one is provided, then the number of variables and clauses MUST match the header.

3. All clauses, with the exception of the last clause, must end with '0', and can extend multiple lines.

4. All variables are numbered 1 - N, where N is the number of variables. Ie, if you say 10 variables, they must be labeled 1, 2, ..., N. And not 2, 3, ..., 11 or any other convention.

Thus, valid input can take on many forms:

$$\begin{array}{l} \text{p cnf 4 2} \\ 1\ 2\ -3\ 0 \\ -1\ 4\ 0 \end{array}$$

[2] https://jix.github.io/varisat/manual/0.2.0/formats/dimacs.html

```
4 2
1 2 -3 0
-1 4 0
```

```
p cnf 4 2
c this comment doesn't have to be at the top
1 2 -3 0
-1 4 0
```

```
p cnf 4 2
c this comment doesn't have to be at the top
1 2 -3 0
-1 4
```

```
c the first two numbers are still the number
c of variables and the number of clauses
p cnf 4 2 1 2 -3 0
-1 4 0
```

3 Brute Force

The absolute simplest way of solving a SAT problem. In this section, we create our base SAT solver from scratch. We will not add any fancy features such as incremental solving, backtracking, or even basic contradiction detection... at first. In our simple brute force solver, we are given an input as defined above, and we simply solve it as a CNF or DNF equation by checking every possible combination of variable assignment.